

Performance Analytics and Delay Prediction of Public Transportation (Boston and Seattle)

Course: EAS 587 Data Intensive Computing (Fall 2025)

Achisman Sarangi¹, Nupur Srivastava², Sarthak Sharma³

School of Engineering and Applied Sciences

University at Buffalo, SUNY

Buffalo, NY, USA

{achisman, nupursri, ss694}@buffalo.edu

Abstract—This project phase transitions the Seattle Bus Delay analysis from local processing to large-scale distributed computing and AI integration. Utilizing Apache Spark (PySpark), we processed over 60 million transit records, integrating a secondary weather dataset to enhance predictive capabilities. We implemented an end-to-end pipeline including data cleaning, feature engineering using Window functions, and training three regression models (Linear Regression, Random Forest, and GBT). The Linear Regression model was selected for deployment due to superior performance ($R^2 = 0.76$) and computational efficiency. Finally, we developed a Model Context Protocol (MCP) server to expose these analytics to an AI assistant (Claude), culminating in an interactive React-based dashboard for real-time delay visualization. This phase additionally extends the system to Bostons MBTA subway network, integrating static GTFS schedules, real-time train movement data, and contextual sources such as weather, service alerts and city events. Apache Spark enabled large-scale ingestion, cleaning, and feature engineering for millions of subway records. A two-stage modeling approach delay classification followed by regression was implemented to address Bostons non-linear, disruption-driven operational patterns. MCP integration further enabled AI-driven querying and model serving for both cities, demonstrating real-time interpretability and end-user accessibility. Overall, this project presents a scalable, multi-city transit analytics pipeline that shifts from static timetables to dynamic, AI-driven risk intelligence, supporting more informed decisions for commuters, transit agencies, and city planners.

Index Terms—GTFS, public transportation, Spark, delay prediction, MCP, real-time analytics, machine learning

I. INTRODUCTION

Urban transit reliability is a critical component of modern city infrastructure, directly affecting commuter experience, operational efficiency, and long-term planning. In earlier project phases, our analysis of Seattles bus system revealed that delays are strongly influenced by temporal, spatial, and environmental factors. However, Bostons subway network (MBTA) exhibits a different operational profile, where delay patterns are more heavily impacted by headway instability, construction activities, and service disruptions. These differences require a broader, multi-city analytical framework capable of capturing both bus-based and rail-based variability at scale.

Phase 3 expands the scope of the project in four key directions. First, **Scalability**: migrating local workflows to

Apache Spark to efficiently process tens of millions of transit records from both cities. Second, **Multi-Source Integration**: augmenting Seattles analysis with weather data while incorporating Bostons richer contextual sources, including service alerts, construction flags, and major city events, all of which contribute significantly to subway delay behavior. Third, **AI Accessibility**: developing a Model Context Protocol (MCP) server that enables Large Language Models (LLMs) to query transit data, retrieve metrics, and run predictions using natural-language commands. Finally, **Deployment**: creating an interactive dashboard capable of visualizing predicted delays in real time.

After integrating static datasets with real-time feeds and additional secondary sources, we created a unified multi-city pipeline for Seattles King County Metro buses and Bostons MBTA subway, thereby overcoming the limitations of traditional static schedule analysis.

II. METHODOLOGY (SPARK+MCP)

This project implements a unified large-scale Spark pipeline for two transit systems: Seattles King County Metro buses and Bostons MBTA subway network. Although each system exhibits distinct operational characteristics, both follow the same general workflow of ingestion, cleaning, feature engineering, machine learning, and AI-based deployment through MCP.

A. Data Sources

1) Seattle Data Sources: **Primary Sources**:

- **Static GTFS (King County Metro)**: Scheduled routes, trips, stop times, and shapes.
- **GTFS-Realtime Bus Feeds**: Seattle Real-Time Firehose Data (bus locations and delays).

Secondary Sources:

- **Weather History**: hourly temperature and precipitation

2) Boston Data Sources: **Primary Sources**:

- **Static GTFS (MBTA Subway)**: Route definitions, stop sequences, trip schedules, and frequency information.
- **GTFS-Realtime Subway Feeds (LAMP Archive)**: Historical subway vehicle movements including actual arrival and departure timestamps.

Secondary Sources:

- **Weather Data:** Daily weather attributes aligned by service date.
- **Service Alerts:** Route-level disruptions, construction events, outages, and delay advisories.
- **City Event Data:** Records of major Boston events associated with crowd surges and demand anomalies.

Static and historical datasets were stored in Parquet format, while real-time and contextual sources were ingested dynamically through API retrieval.

B. Spark Data Engineering Pipeline

Apache Spark was used for all large-scale transformations across both cities. The pipeline consists of the following core stages:

1) 1) Cleaning and Standardization:

- Normalization of timestamps into consistent formats.
- Removal of malformed or duplicated records.
- Outlier filtering based on operational thresholds (e.g., unrealistic early or late arrivals).

This ensures reliability and consistency across multimodal datasets.

2) 2) Feature Extraction: Common features extracted for both cities include:

- hour_of_day, day_of_week, is_peak_hour
- trip- and route-level identifiers
- stop-level temporal alignment

Boston additionally required computation of scheduled headways using GTFS frequency rules, while Seattle emphasized stop-sequencebased timing.

3) 3) Window-Based Analytics: Window functions were central to constructing sequential and historical features.

Seattle (Lag-Based Delay):

```
windowSpec = Window.partitionBy("trip_id").
    orderBy("stop_sequence")
df = df.withColumn("prev_delay", lag("
    arrival_delay").over(windowSpec))
```

III. RESULTS AND ANALYSIS

This section presents model performance and analytical insights for both Seattle's bus network and Boston's subway system. Although both cities used a unified Spark engineering pipeline, the modeling strategies and outcomes differed due to the distinct operational characteristics of each transit system.

A. Seattle: Regression Performance

Three regression models were evaluated using an 80/20 train/test split. Due to memory constraints, tree-based models were trained on a 10% subsample, while Linear Regression was trained on the full dataset. Table I reports the evaluation metrics.

The Linear Regression model significantly outperformed the tree-based models. This is attributed to (1) the strong linear correlation between `prev_delay` and `arrival_delay`, and (2) the fact that Linear Regression trained on the full

TABLE I
SEATTLE MODEL EVALUATION METRICS

Model	Train Time (s)	RMSE (s)	R^2
Linear Regression	443.04	117.98	0.7640
Gradient Boosted Trees	660.45	185.08	0.4194
Random Forest	533.20	190.23	0.3866

dataset rather than a sampled subset. These results validate that Seattle's delay patterns are approximately linear and strongly dependent on the delay observed at the previous stop.

1) *Seattle Key Insights:* Spark SQL analysis revealed meaningful behavioral patterns:

- **Rain Impact:** Light rain (< 0.3 mm/hr) has negligible effect, while heavy rain (> 0.4 mm/hr) increases average delay from ~ 63 seconds to 131 seconds.
- **Rush Hour:** Peak congestion occurs at 9:00 AM (99.5 s) and at 1:00 PM (97.1 s), the latter likely due to reduced night service frequency.
- **Temperature Effects:** Mild weather (51°C) showed higher average delays (66 s) than cold weather ($< 5^\circ\text{C}$, 45 s), possibly reflecting higher ridership during pleasant conditions.

These insights highlight the value of incorporating weather and temporal patterns when modeling transit delay behavior.

B. Boston: Two-Stage Classification and Regression

Boston's subway system exhibits greater operational variability, heavily affected by headway instability, construction events, and service alerts. To address this, a two-stage modeling pipeline was developed.

1) *Stage 1: Delay Classification:* A Weighted Random Forest classifier outperformed Logistic Regression and Gradient Boosting Regressor. Applying a $5.6\times$ weight to the minority class improved sensitivity to rare delays.

- **AUC:** 0.683
- **Precision:** 0.813
- **Recall:** 0.561

Fig. 1 shows the confusion matrix, illustrating the classifiers improved ability to detect delayed trips after reweighting.

2) *Stage 2: Delay Magnitude Regression:* Standard regression models trained on the entire dataset yielded poor results ($R^2 < 0.10$) due to the heavy imbalance of on-time trips. The two-stage approach, which trained a specialist Random Forest regressor solely on the subset of delayed trips identified by the classifier, significantly improved predictive capability, achieving:

- **Two-Stage Regression:** $R^2 \approx 0.144$

This may be expected given Boston's non-linear, disruption-heavy dynamics. The relative importance of engineered features is shown in Fig. 2, highlighting the dominant role of headway-based predictors.

C. Comparative Insights Across Cities

Several patterns emerged across the two systems:

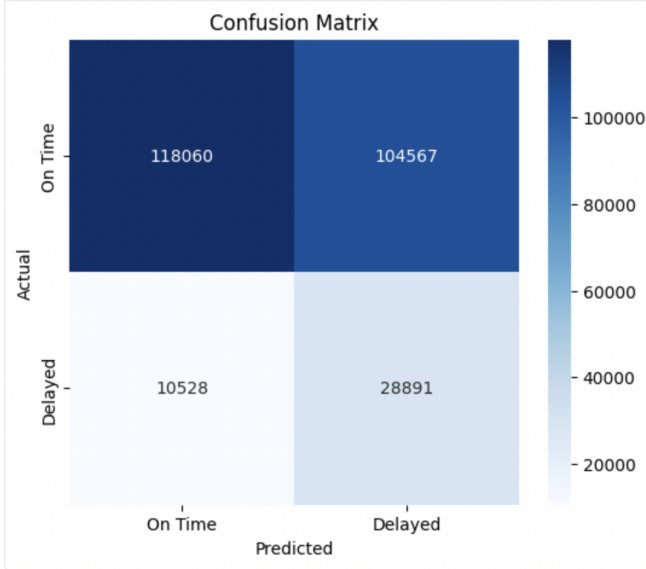


Fig. 1. Confusion matrix for the Boston weighted Random Forest delay classifier. Reweighting improves recall by reducing missed delays (false negatives).

- **Predictive Stability:** Seattle exhibited stronger linear relationships, enabling high regression accuracy. Boston required classification first due to the higher probability of zero-delay and strong non-linear effects.
- **Headway Effects:** Boston delay patterns were dominated by headway instability, rolling headways and lag features were top predictors.
- **Contextual Importance:** Weather affected both cities, while service alerts and events were particularly influential in Boston.
- **Modeling Strategy:** The unified Spark pipeline enabled consistent scaling, but city-specific modeling approaches captured operational differences effectively.

D. MCP-Enabled Deployment and Visualization

1) *Seattle: MCP Prediction Example:* The MCP integration allows natural-language queries to trigger real-time delay predictions from the Seattle regression model. Users specify conditions such as route selection, day-of-week, temperature, rainfall, and the LLM forwards these parameters to the MCP tool, which returns the predicted delay in seconds. An example interaction is shown in Fig. 3.

2) *MCP Server Integration:* The predictive pipeline is exposed to the LLM through a local MCP server named `seattle-bus`. This server registers Spark-based model inference functions and data access tools, making them discoverable and callable from within Claude. Fig. 4 shows the MCP server configuration in the Claude Desktop client.

3) *Interactive Dashboard and Temporal Insights:* We also successfully developed an interactive dashboard that lets users explore model behavior under different scenarios. As shown in Fig. 5, users can adjust rainfall, temperature, day-of-week, and route selection to observe the corresponding delay predictions.

This demonstrates how the underlying Spark models can be surfaced as a user-friendly, real-time decision support tool.

The dashboard also supports simulation across the 24-hour service window. Fig. 6 shows the predicted delay curve by hour of day under fixed weather conditions, revealing clear morning and evening peak windows that are consistent with the patterns identified in the Spark-based analysis.

IV. DISCUSSION OF FINDINGS, LIMITATIONS, AND FUTURE WORK

1) *Findings:* For Seattle, the most significant finding is the strong predictive power of the lag-based feature (`prev_delay`). This confirms that delays propagate downstream: once a bus is late at an upstream stop, it rarely recovers time over the remainder of the route. This effect explains why Linear Regression, driven largely by this single feature, achieved high predictive accuracy despite model simplicity.

For Boston, headway instability emerged as the strongest predictor of delay risk. Variability in both scheduled and real-time headways captured the operational irregularities of the MBTA system. In addition, service alerts and construction-related disruptions significantly increased the likelihood of delays, highlighting the importance of contextual operational data in modeling subway reliability.

2) *Limitations:* Despite the overall effectiveness of the unified pipeline, several limitations constrained model performance and deployment capabilities.

Resource Constraints: Training computationally intensive models such as Gradient Boosted Trees and Random Forests on more than 60 million records exceeds the capacity of a single local machine. As a result, these models were trained on a 10% subsample, which reduced their predictive accuracy relative to the full-data Linear Regression model. A distributed environment (e.g., Databricks or AWS EMR) would mitigate these constraints.

MCP Latency: Initializing the Spark context within the MCP server introduces a startup latency of 5 to 10 seconds.

Seasonal Data Availability: The analysis was limited by the availability of only a partial-season dataset. Delay behavior is known to vary across seasons due to weather, construction cycles, and ridership changes. The absence of multi-season data restricts the generalizability of the models and their ability to capture long-term delay patterns.

3) *Future Work:* Future extensions of this project will focus on improving both the predictive capability and operational deployment of the system. First of all, future iterations would involve deploying the Spark cluster on cloud infrastructure (Databricks) to train the GBT model on the full dataset.

Additionally, the MCP server could be connected to a live API endpoint rather than a static Parquet file for real-time inference. Finally, incorporating richer data sources such as maintenance logs, traffic density data, and infrastructure breakdowns would provide a more comprehensive view of the factors influencing delay patterns.

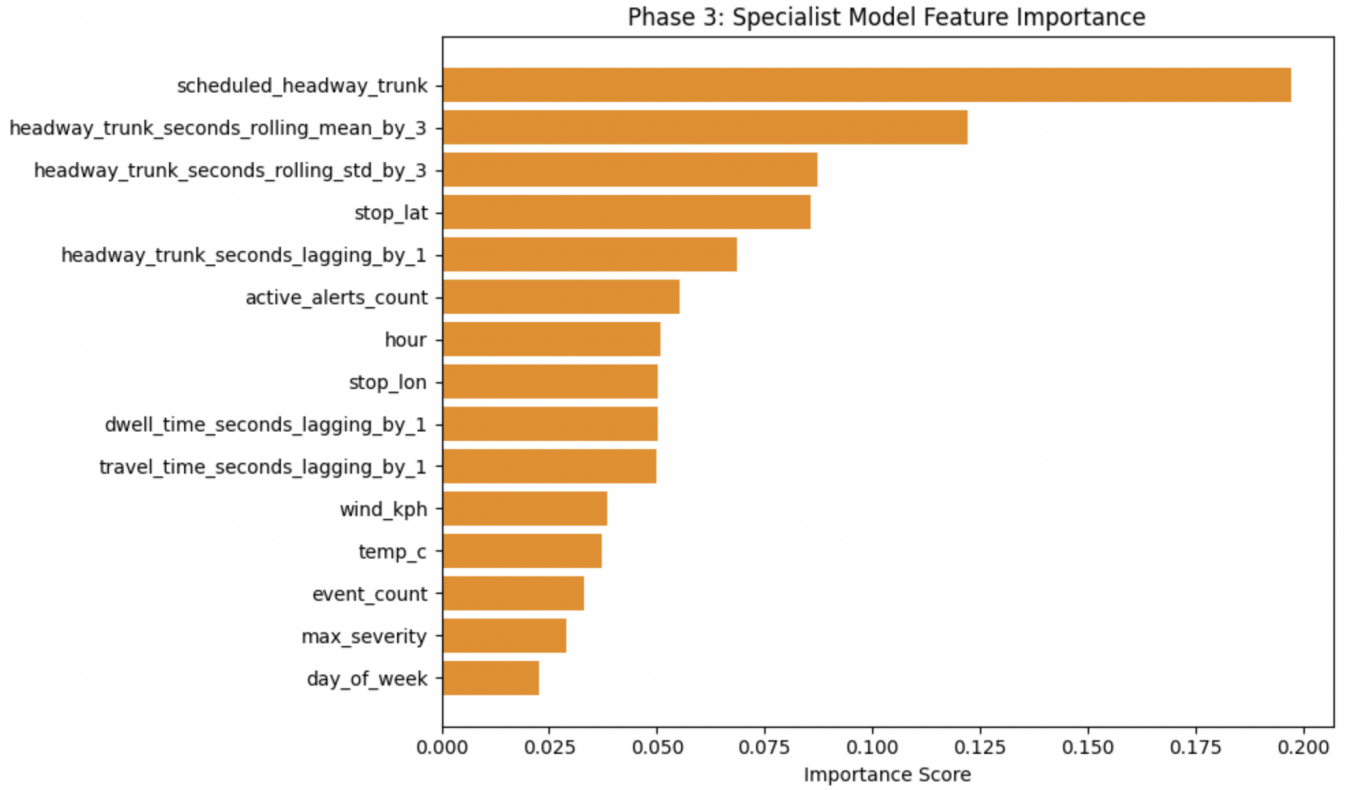


Fig. 2. Feature importance scores for the Boston specialist regression model. The most influential predictors include scheduled headway, rolling headway variability, and alert-related variables.

V. CONCLUSION

Phase 3 successfully demonstrated the scalability of the data science pipeline. By migrating to Apache Spark, we processed data volumes that would crash standard tools like Pandas. Integrating the Model Context Protocol (MCP) transformed the static model into an interactive "Data Product," allowing users to query the system using natural language. The final dashboard effectively visualized the correlation between weather conditions and transit delays, fulfilling the project's use case requirements.

REFERENCES

- [1] Apache Spark Documentation, MLlib: Main Guide. [Online]. Available: <https://spark.apache.org/docs/latest/ml-guide.html>
- [2] Model Context Protocol, Introduction to MCP. [Online]. Available: <https://modelcontextprotocol.io/>
- [3] Seattle Department of Transportation, Real-Time Firehose Data, 2025.
- [4] Massachusetts Bay Transportation Authority, MBTA Developer Portal: GTFS Feeds. [Online]. Available: <https://www.mbta.com/developers/gtfs>
- [5] Open-Meteo, Open-Meteo Weather API: Free Weather Forecast and Historical Data. [Online]. Available: <https://open-meteo.com/>
- [6] City of Boston, "Special Event License Applications." [Online]. Available: <https://data.boston.gov/dataset/special-event-license-applications>

APPENDIX A

APPENDIX: CODE SNIPPETS (SEATTLE)

A. MCP Server Configuration (*server.py*)

```

from mcp.server.fastmcp import FastMCP
from pyspark.ml import PipelineModel
from pyspark.sql import SparkSession

# Initialize MCP server
mcp = FastMCP("SeattleBusDelayServer")

# Start Spark session
spark = SparkSession.builder \
    .appName("SeattleBusDelayMCP") \
    .getOrCreate()

# Load trained Linear Regression model
model = PipelineModel.load("./models/
    linear_regression")

@mcp.tool()
def predict_bus_delay(hour: int, rain: float,
    prev_delay: float, temp_c: float):
    """Predicts bus delay based on time and
    weather conditions."""
    data = spark.createDataFrame(
        [(hour, rain, prev_delay, temp_c)],
        ["hour", "rain", "prev_delay", "temp_c"]
    )
    result = model.transform(data).head().
        prediction
    return f"Predicted_Delay: {result:.2f}
        seconds."

```

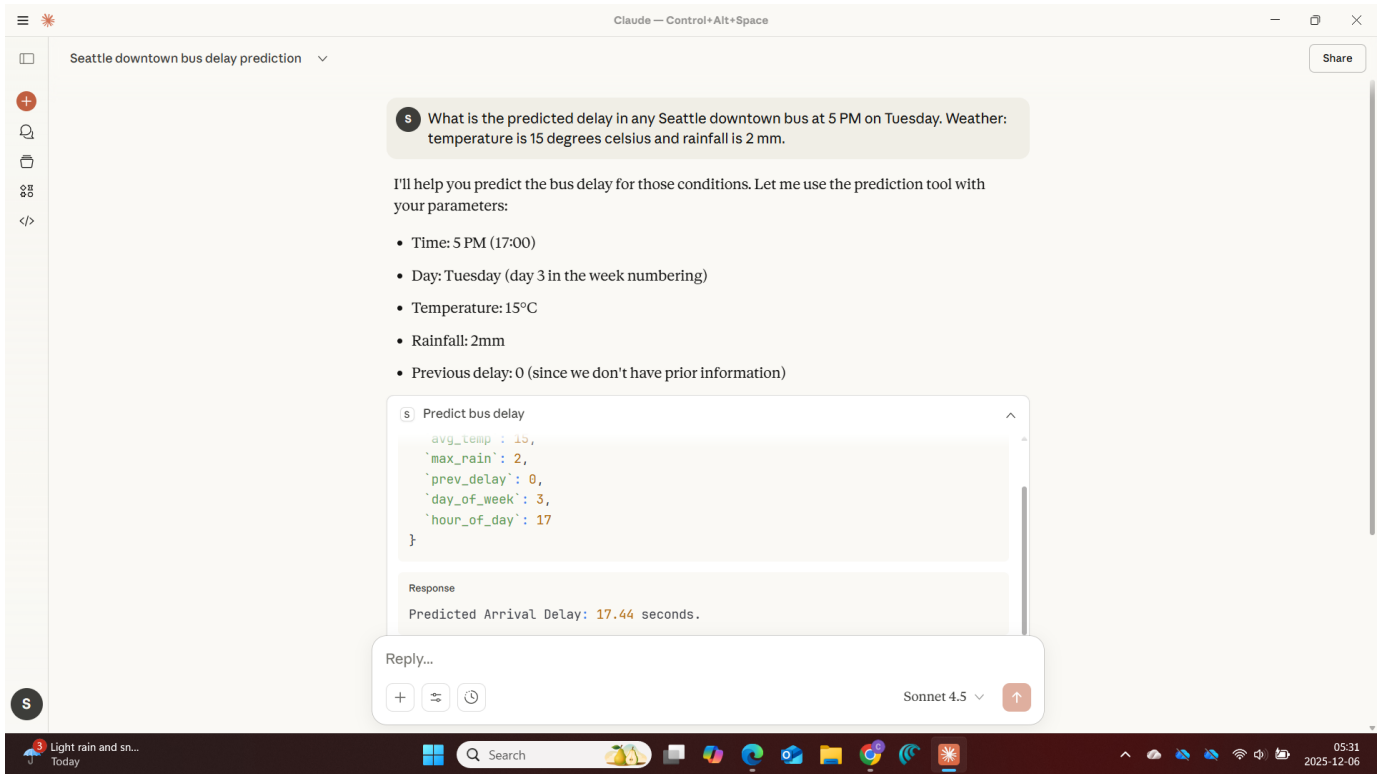


Fig. 3. MCP-powered prediction inside Claude for Seattle buses. The LLM sends structured parameters (route selection, day-of-week, temperature, rainfall) to the MCP tool, which returns a real-time delay estimate.

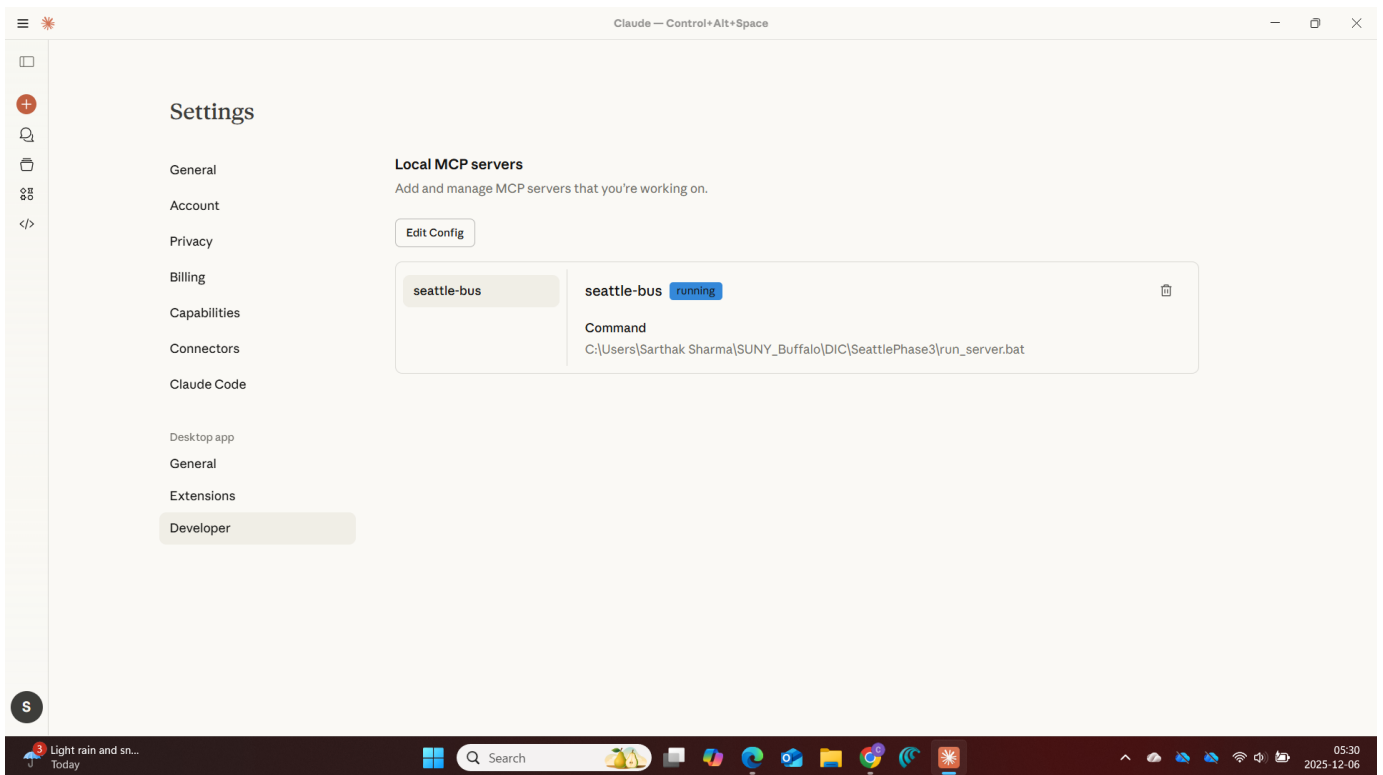


Fig. 4. Local MCP server (seattle-bus) configured and running inside the Claude Desktop application. The server exposes Spark ML models and data queries as MCP tools for use by the LLM.

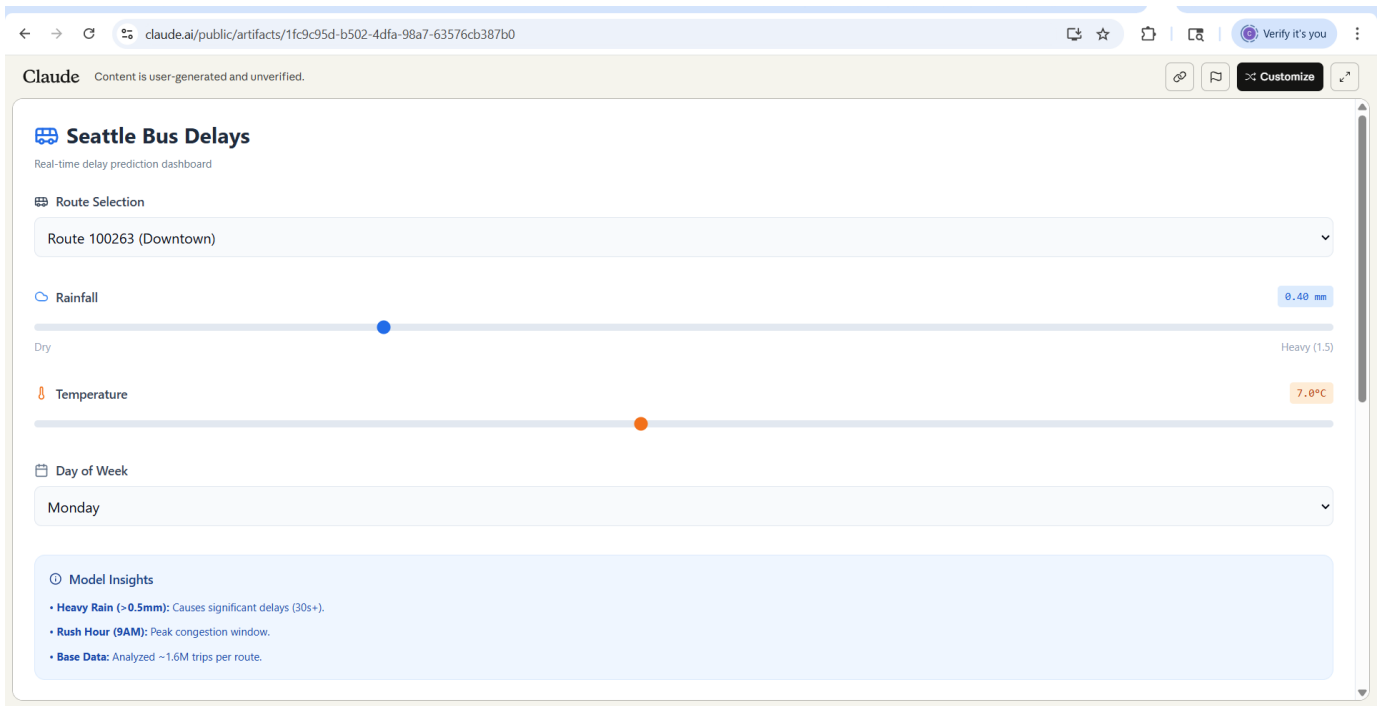


Fig. 5. Interactive Seattle bus delay dashboard generated via MCP and Claude. Sliders allow users to modify rainfall, temperature, and day-of-week to visualize predicted delay conditions for downtown routes.

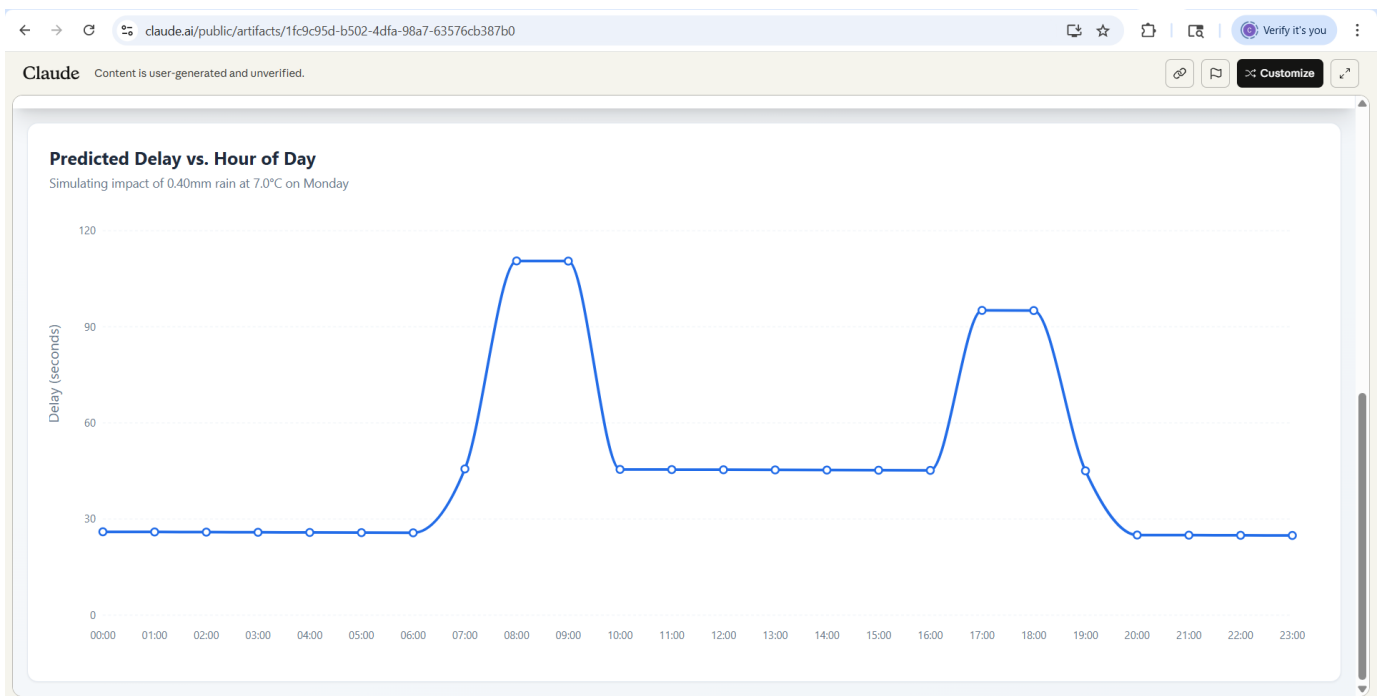


Fig. 6. Predicted delay vs. hour of day, visualized through the MCP-enabled dashboard. The model captures pronounced morning and evening rush-hour peaks under the specified weather scenario.

B. Spark Window Function for Lag Feature

```
from pyspark.sql.window import Window
from pyspark.sql.functions import lag
```

```
# Create window ordered by stop sequence for
# each trip
windowSpec = Window.partitionBy("trip_id").
    orderBy("stop_sequence")
```

```
# Seattle: Construct lag-based delay
propagation feature
df_bus = df_bus.withColumn(
    "prev_delay",
    lag("arrival_delay").over(windowSpec)
)
```

APPENDIX B

APPENDIX: CODE SNIPPETS (BOSTON)

A. Boston History-Based Feature Engineering

```
# Rolling-history window: previous up to 3
observations
w_hist = (
    Window.partitionBy("service_date", "
        route_group", "stop_id")
        .orderBy("order_s")
        .rowsBetween(-3, -1)    # look back
                                over the last 3 records
)
# Simple lag window (1-step lag)
w_lag1 = (
    Window.partitionBy("service_date", "
        route_group", "stop_id")
        .orderBy("order_s")
)
```

B. Class Weight Calculation for Imbalanced Delay Data

```
# Count positives (delayed) and negatives (on-
time)
pos_count = train_df.filter(F.col(target_cls)
    == 1).count()
neg_count = train_df.filter(F.col(target_cls)
    == 0).count()

# Compute imbalance ratio
balance_ratio = neg_count / pos_count
print(f"Weight_Ratio:_{balance_ratio:.2f}")

# Apply class weights: delayed trips receive
higher weight
train_df = train_df.withColumn(
    "classWeights",
    F.when(F.col(target_cls) == 1,
        balance_ratio).otherwise(1.0)
)
```

C. Weighted Random Forest Classifier

```
assembler = VectorAssembler(
    inputCols=feature_cols,
    outputCol="features"
)
rf = RandomForestClassifier(
    labelCol="is_delayed",
    featuresCol="features",
    weightCol="class_weight",
    numTrees=200
)
pipeline = Pipeline(stages=[assembler, rf])
rf_model = pipeline.fit(train_df)
```

D. Specialist Regression Model for Delayed Trips

```
# Filter training data (only delayed trips)
train_df_delays_only = train_df.filter(
    (F.col("is_delayed") == 1) & F.col("
        delay_seconds").isNotNull()
)
test_df_delays_only = test_df.filter(
    (F.col("is_delayed") == 1) & F.col("
        delay_seconds").isNotNull()
)
# Train Random Forest regressor
rf_reg = RandomForestRegressor(
    labelCol="delay_seconds",
    featuresCol="features",
    numTrees=100,
    maxDepth=10,
    seed=42,
)
rf_model_specialist = rf_reg.fit(
    train_df_delays_only)
# Evaluate
predictions = rf_model_specialist.transform(
    test_df_delays_only)
evaluator_r2 = RegressionEvaluator(
    labelCol="delay_seconds", predictionCol="
        prediction", metricName="r2"
)
print("R2 =", evaluator_r2.evaluate(
    predictions))
```